124. Binary Tree Maximum Path Sum

PLATFORM	DIFFICULTY	DATE
Platform: LeetCode	HARD	Solved: July 31, 202

Problem Summary

Given the root of a binary tree, return the maximum path sum of any non-empty path. A path can start and end at any node and doesn't need to pass through the root.

Approach

Post-order DFS — at each node compute the best path through it (left + right + root.val) and update global max. Return only the best single arm (root.val + max(left, right)) to parent since a path can't split.

Algo

1. Initialize maxPath = Integer.MIN_VALUE.
2. In solve: if root == null return 0.
3. left = Math.max(0, solve(root.left)) — ignore negative contributions.
4. right = Math.max(0, solve(root.right)).
5. maxPath = Math.max(maxPath, left + right + root.val).
6. Return root.val + Math.max(left, right).

Key Insights

- Math.max(0, solve(...)) — if subtree contributes negatively, treat it as 0 (don't include it).
- Function does two jobs: updates global max (full path) and returns single arm (for parent).
- Integer.MIN_VALUE not 0 as initial max — tree can have all negative values.
- Path through node = left + right + root.val; arm returned to parent = root.val + max(left, right).

Time Complexity & Space Complexity

O(n) — every node visited once.

O(h) — recursion stack.

Observations

- Integer.MIN_VALUE as initial value handles all-negative trees correctly — 0 would give wrong answer.
- Math.max(0, ...) for left/right neatly handles negative subtrees — no if/else needed.
- Single arm returned to parent because a path can't fork — this is the key constraint.